

Executive Summary

- **Data Flow:** RTDSClnt pushes BTC/ETH market data into `ArbEngine._onPriceUpdate()`. This routine (1) updates Polymarket data, (2) sets the current “K” (opening price) via **BayesianEstimator**, (3) computes the trade edge via **EdgeCalculator**, and (4) logs signals and executes (paper) trades. A Mermaid flowchart below maps this end-to-end flow.
- **Dependencies:** Key modules are `RTDSClnt`, `PolymarketFetcher`, `BayesianEstimator`, `EdgeCalculator`, `KellySizer`, `TelegramControl`, etc. Each is instantiated in the `ArbEngine` constructor and drives the logic (e.g. Binance/Chainlink via RTDS, Polymarket data, Bayesian probability, etc.).
- **Timestamps:** Multiple clocks are used. `polyData.windowTs` (from `PolymarketFetcher`) drives window alignment, `rtdsState.chainlinkAge` (ms) comes from RTDS, and `Date.now()` is used internally for K-setting and staleness checks. These are **not unified** – e.g. window rollover uses Polymarket time, while vacuum detection uses system time. This mismatch can lead to subtle edge cases.
- **Age/Price Variables:**
- **BTC side:** `rtdsState.chainlinkPrice` and `rtdsState.chainlinkAge` feed into `BayesianEstimator`. The estimator’s `openingPrice` is set once per 5m window (the “K”). It also tracks `priceHistory` (timestamped via `Date.now()`). `polyData.spread`, `.bidDepth`, etc. provide the orderbook state.
- **ETH side:** Similarly uses `rtdsState.ethChainlinkPrice` / `ethChainlinkAge` and a separate `this.ethBayesian` with its own `openingPrice`. `PolymarketFetcher`’s ETH data (`ethData.windowTs`, `priceUp`, etc.) drive the ETH flow.
- **Control Flow:** Many early-return branches exist: e.g. pause flag, missing Polymarket data, no Chainlink price, stale K (Bayesian returns null), etc. ETH is processed independently (but in the same tick function). A critical asymmetry was found: on a stale-K for ETH, the code does `return; // skip this ETH tick`, which aborts the entire function (skipping any remaining logic). Similarly, a null BTC `bayesResult` triggers `return`, skipping ETH processing.
- **Suppression Flags vs Execution:** The code builds `suppressionFlags` arrays (e.g. `['warmup', 'trend', ...]`) but **only the BTC path applies them in the DB insert**. The ETH signal insert (as written) does *not* supply suppression flags or imbalance fields, a clear bug (see Table below).
- **Signals/PaperWrites:** The `signals` table logs each tick’s state (raw/net edge, filters, flags, prices). The `paper_trades` table logs each executed trade (timestamp, side, entry, stake, pre-penalty edge, etc.). We verified that **the values stored match the code’s intent** (e.g. `net_edge_pct` = `edgeResult.netEdge*100` from pre-trade values). One inconsistency: the signals insertion for ETH omits many columns (suppression flags, `raw_edge_pct`, etc.), which must be fixed.
- **Logging Gaps:** Logging is extensive (resolutions, balances, trades), but it misses a few cases. In particular, there is no log when a stale tick causes *any* skip except the console warning for deferred resolution. We suggest adding logs for skipped ticks (e.g. “Skipped BTC tick: no Chainlink price”, or “ETH tick skipped: stale K”). Also, `console.error` catches on `onPriceUpdate`, but adding a log on `return null` (stale K) would aid debugging.

Top 10 Recommended Fixes (with rough timeline):

#	Issue & Severity	Location	Fix Summary
1	ETH signal insert mismatch (Critical): many <code>signals</code> columns (<code>suppression_flags</code> , <code>raw_edge_pct</code> , <code>final_edge_pct</code> , <code>trend_penalty</code> , <code>trend_would_block</code>) are not provided for ETH.	<code>arb_engine.js</code> (ETH signal insert block, around lines 381–420)	Supply the missing fields for ETH <code>insertSignal.run(...)</code> exactly as done for BTC. I.e., append <code>ethSuppressionFlags</code> , <code>ethImbalanceRatio</code> , <code>ethImbalanceDelta</code> , <code>ethImbalanceBucket</code> , <code>ethWouldTradeDown</code> , <code>ethWouldTradeUp</code> , <code>ethNearThreshold</code> , <code>ethEdge.rawEdge*100</code> , <code>(ethEdge.netEdge*ethTrendPenalty)*100</code> , <code>ethTrendPenalty</code> , <code>ethTrendBlocked?1:0</code> in the parameter list. (Or better, share a code block snippet below)
2	Chainlink age default (High): In <code>BayesianEstimator</code> , the call <code>recordOpeningPrice(currentPrice, windowTs, chainlinkAgeMs ?? 0)</code> silently treats an undefined age as 0 (fresh). If RTDS ever fails to supply an age, stale K would be accepted.	<code>bayesian.js</code> , in <code>estimate()</code> before <code>recordOpeningPrice</code>	Require explicit age. For example, change to <code>const age = (chainlinkAgeMs !== undefined ? chainlinkAgeMs : null)</code> ; <code>const ok = this.recordOpeningPrice(currentPrice, windowTs, age);</code> and handle null accordingly (or simply assume RTDS always supplies it). At minimum, log a warning if age is undefined before treating as 0.
3	Unintended full-skip on ETH error (High): The ETH path does <code>if (!ethBayes) return;</code> . This <code>return</code> aborts the entire <code>_onPriceUpdate</code> (skipping any remaining logic) – but BTC processing already ran. If an ETH tick has stale K, it may unnecessarily stop logging or other trailing tasks.	<code>arb_engine.js</code> , around line 316 (inside ETH path)	Change <code>return;</code> to e.g. <code>continue;</code> or better restructure to only skip ETH-specific logic. Since this is inside an <code>async</code> method, replace with <code>{ /* do nothing further for ETH */ }</code> or factor ETH code into its own function so that a return doesn't skip the rest. Ensure processing and logging continues even if ETH bayesian returns null.
4	Uncaught <code>chainlinkPrice</code> null (Medium): Immediately after <code>const chainlinkPrice = rtdsState.chainlinkPrice;</code> , the code does <code>if (!chainlinkPrice) return;</code> . If <code>0</code> is a possible valid price, it would skip incorrectly; if <code>undefined</code> , okay. But no log is made.	<code>arb_engine.js</code> , line ~515	Explicitly log when skipping: e.g. <code>if (!chainlinkPrice) { console.log("[Engine] Skipping tick with no BTC Chainlink price"); return; }</code> . Confirm whether 0 is impossible (likely yes).

#	Issue & Severity	Location	Fix Summary
5	Eth suppression flags unused (Medium): The ETH signal insertion never uses <code>ethSuppressionFlags</code> (defined at line 374–379). A <code>suppression_flags</code> field in the table will remain null/incorrect for ETH signals.	<code>arb_engine.js</code> , ETH signal insert (around lines 374–420)	Include <code>ethSuppressionFlags</code> in the <code>insertSignal.run(...)</code> arguments, analogous to BTC's <code>suppressionFlags</code> . Without it, debugging ETH suppression will break.
6	Vacuum/Post-vacuum fields for ETH (Medium): The BTC insert populates <code>post_vacuum</code> and <code>vacuum_age_s</code> , but the ETH insertion as shown only has a <code>0 // no ETH lag detection</code> . The <code>post_vacuum</code> column likely defaults to 0/null for ETH.	<code>arb_engine.js</code> , ETH signal insert (lines 381–420)	Add ETH versions of those fields (<code>ethEdge.postVacuum</code> , <code>(Date.now() - this.lastVacuumTime)/1000</code> if in vacuum etc.), to mirror the BTC pattern.
7	Window alignment (Medium): <code>polyData.windowTs</code> is used as the source of truth for the 5m window, but there's no check of RTDS time vs window. If RTDS ticks arrive slightly early/late, the first window tick might not actually align.	<code>arb_engine.js</code> , rollover block (around line 100)	(Diagnostic) Log a warning if <code>windowTs</code> changes too early/late relative to <code>secondsRemaining</code> . Possibly adjust logic to K-setting on <code>windowTs</code> instead of RTDS tick arrival, or vice versa.
8	Time source confusion (Low): Many <code>Date.now()</code> usages (vacuum, <code>lastPolyFetch</code> , <code>priceHistory</code>) coexist with Polymarket and RTDS time. E.g., <code>Date.now()</code> vs <code>windowTs</code> units mismatch.	<code>bayesian.js</code> and <code>arb_engine.js</code> (various)	Ensure all timestamps are in a consistent epoch (e.g. always ms). Document that <code>windowTs</code> is ms since epoch. Add comments or convert if necessary. For logging, prefer ISO.
9	Logging gaps (Low): No log is emitted when a tick is skipped due to pause, no data, or bayesian null (BTC). This hampers debugging.	<code>arb_engine.js</code> , at each early return	Insert brief <code>console.log</code> or debug when tick is skipped (e.g. after <code>if (this.pause) return;</code> , after <code>if (!polyData) return;</code> after <code>if (!bayesResult) return;</code>). Use consistent formatting.
10	Signal/trade DB consistency (Low): Confirm that <i>pre</i> -penalty values are captured. In <code>signals</code> , <code>raw_edge_pct</code> and <code>final_edge_pct</code> are stored <i>post</i> - and <i>pre</i> -trend respectively. In <code>paper_trades</code> , only <code>net_edge_pct</code> is stored (assumed pre-trend). Verify that intention is correct; document it.	Documentation	(No code change) Add comments or a line in code noting that <code>net_edge_pct</code> in <code>paper_trades</code> is <i>before</i> trend penalty. If needed, log both to DB.

<div style="page-break-after: always;"></div>

Data Flow Diagram

```
graph LR
    A[RTDS WebSocket<br>(binancePrice, chainlinkPrice, ages)] -->|update event| B[_onPriceUpdate(state)]
    B --> C[PolymarketFetcher (BTC, ETH)]
    C -->|windowTs, secLeft, orderbook| B
    B --> D[BayesianEstimator (BTC & ETH)]
    D --> E[EdgeCalculator (BTC & ETH)]
    E --> F[Signal Table (db)]
    E --> G[Trade Decision]
    G --> H[PaperTrades Table (db)]
    H --> I((End))
```

Figure: Each RTDS update (binance & Chainlink) triggers `_onPriceUpdate`, which fetches market data, calls `BayesianEstimator.estimate()`, then `EdgeCalculator.evaluate()`. The result either logs a signal or executes a trade (via KellySizer), which is written to `signals` and `paper_trades` tables respectively.

External Dependencies and Imports

- **Node Modules:** Uses `events`, `better-sqlite3`, `path`, `fs`, and `node-fetch`.
- **Local Modules:**
 - `RTDSClient` (in `./rtds_client`): subscribes to Polymarket RTDS (resolves to binance/chainlink updates).
 - `PolymarketFetcher` (in `./polymarket_fetcher`): fetches market data (up/down prices, spreads, depths).
 - `BayesianEstimator` (in `./bayesian.js`): computes fair probability from price history.
 - `EdgeCalculator` (in `./edge_calculator`): calculates net edge (including trend penalty).
 - `KellySizer` (in `./kelly`): computes position sizing (Kelly criterion).
 - `TelegramControl` (in `./telegram_control`): handles alert messages.
 - `SpreadModel` (in `./spread_model`): tracks spread-based signals (used for diagnostics only).

Each module plays a role in the flow (e.g. RTDS and PolymarketFetcher feed data to BayesianEstimator and EdgeCalculator). There are no obvious circular dependencies. All imports are at the top of `arb_engine.js`.

Time Sources and Timestamps

1. `rtdsState` **Timestamps:**
2. `rtdsState.chainlinkAge` (ms): Time since last Chainlink price update (set by RTDSClient).
3. `rtdsState.binanceAge` (ms): Time since last Binance update (not directly used here).
4. `rtdsState.secondsRemaining` and `windowTs`: Passed from `PolymarketFetcher` – likely this `windowTs` is the epoch milliseconds of the window start (or end). We assume it is consistent (but this should be confirmed).

5. `Date.now()`: Used within `bayesian.js` (for volatility timestamps) and `arb_engine.js` (for vacuum timing and throttle).

6. Possible Mismatches:

7. The code uses Polymarket's `windowTs` (likely epoch) and also uses `Date.now()`. If PolymarketFetcher's `windowTs` is in seconds or ms, mixing them could cause subtle shifts.
8. For example, vacuum start is logged with `Date.now()`, while vacuum end time is also `Date.now()`. But `windowTs` is from Polymarket. No conflict here, but it's important to know all are ms epoch.
9. The code uses `const now = Date.now()` to throttle Polymarket fetches (with a 1000ms TTL). This is consistent internally.

10. Observations:

11. We see no direct "seed money" or wall-clock time beyond these. All writes to DB use `new Date().toISOString()`.
12. Recommendation: **Document units**. For clarity, it's good to confirm `windowTs` unit (likely ms) and note that `chainlinkAge` is ms. Ensure no off-by-1000 errors (sec vs ms).

Price/Age Variables

Variable	Meaning	Set/Updated Where
BTC Side		
<code>rtdsState.chainlinkPrice</code>	Current Chainlink price (resolution oracle)	Updated by RTDSCClient on tick
<code>rtdsState.chainlinkAge</code>	Age of the Chainlink tick (ms since last update)	Updated by RTDSCClient
<code>this.bayesian.openingPrice</code>	"K" = reference price at window start (Chainlink-based)	Set in <code>BayesianEstimator.recordOpeningPrice()</code> on new window

Variable	Meaning	Set/Updated Where
<code>this.bayesian.priceHistory</code>	History of recent K prices (for volatility model)	Populated in <code>recordOpeningPrice</code> and volatility updates
<code>polyData.windowTs</code>	Window identifier (epoch)	Provided by PolymarketFetcher for each window
<code>polyData.secondsRemaining</code>	Seconds left in the window	Provided by PolymarketFetcher
<code>polyData.priceUp</code>	Polymarket UP price (percent or fraction)	Fetches by PolymarketFetcher
<code>polyData.priceDown</code>	Polymarket DOWN price	Fetches by PolymarketFetcher
<code>polyData.spread</code>	Current bid-ask spread (fraction)	Fetches by PolymarketFetcher
<code>polyData.bidDepth</code>	Orderbook bid depth	Fetches by PolymarketFetcher
<code>polyData.askDepth</code>	Orderbook ask depth	Fetches by PolymarketFetcher
ETH Side		
<code>rtDsState.ethChainlinkPrice</code>	ETH Chainlink price (from Polymarket's ETH feed)	Updated by RTDSClient
<code>rtDsState.ethChainlinkAge</code>	Age of ETH Chainlink tick (ms)	Updated by RTDSClient
<code>ethData.windowTs</code>	ETH market window (epoch)	Provided by PolymarketFetcher (ETH UpDown)

Variable	Meaning	Set/Updated Where
<code>ethData.secondsRemaining</code>	Seconds left in ETH window	Provided by PolymarketFetcher (ETH)
<code>ethData.priceUp</code>	ETH market UP price	Fetches by PolymarketFetcher (ETH)
<code>ethData.priceDown</code>	ETH market DOWN price	Fetches by PolymarketFetcher (ETH)
<code>ethData.spread</code>	ETH orderbook spread	Fetches by PolymarketFetcher (ETH)
<code>ethData.bidDepth</code>	ETH bid depth	Fetches by PolymarketFetcher (ETH)
<code>ethData.askDepth</code>	ETH ask depth	Fetches by PolymarketFetcher (ETH)

In both cases, the `openingPrice` in the `BayesianEstimator` is set when `windowTs` first appears. The code resets `bayesian` on window rollovers (lines **110–118** above) to start fresh for each 5m.

Control Flow and Early Returns

The `_onPriceUpdate(rtdsState)` method has many conditional exits. Key branches (with their effect):

- **Pause Check:**

```
if (this.paused) return;
```

Effect: No processing of this tick.

- **Resolve Deferred BTC Window:**

If `this.pendingResolution` is set *and* Chainlink is fresh, it forces resolution of the previous window. (Lines **7–47**). This handles late Chainlink ticks. (Not an “exit” but a branch that resolves earlier signals.)

- **Polymarket Fetch Throttle:**

If `>1s` has passed, triggers `this.polymarket.fetchMarketPrice()` and `this.polymarketEth.fetchMarketPrice()`. After fetching, updates `lastPolyData` / `lastEthData`. (Lines **55–70**.)

- **No Polymarket Data:**

```
const polyData = this.lastPolyData;
if (!polyData) return;
```

Effect: If BTC Polymarket data is unavailable, skip tick entirely.

- **Liquidity Vacuum:**

If the orderbook is empty (`polyData.spread == null`), logs vacuum start or end (Lines **75–88**). Not an exit, but affects `post_vacuum` flags.

- **Window Rollover:**

```
if (this.lastWindowTs && this.lastWindowTs !== windowTs) { ... window
    rolled ... }
```

Effect: Detects new 5m window. It resolves the old window now (immediately, or defers if CL is stale), resets state, and logs. No return here.

- **Chainlink Price Available:**

```
const chainlinkPrice = rtdsState.chainlinkPrice;
if (!chainlinkPrice) return;
```

Effect: If no new BTC Chainlink tick is present in this RTDS update, skip trading logic. (Adds no log.)

- **Bayesian Estimate (BTC):**

```
const bayesResult = this.bayesian.estimate(...);
if (!bayesResult) { return; /* stale K: skip this tick */ }
```

Effect: If K-guard in `bayesian.estimate()` refused to set K (stale tick) or K not set, skip everything. (Log wise, `recordOpeningPrice` logs a K-guard message, but we suggest adding a log here too.)

- **Edge Evaluation:**

After `bayesResult` passes, computes `edgeResult = this.edge.evaluate(...)`. No early exit here – all results (filter failures etc.) are used to decide trading.

- **Signal Insertion (BTC):**

Always runs `insertSignal.run(...)` (even if `edgeResult.tradeable` is false) – thus logs every tick in the signals table with whatever filters/pass flags occurred.

- **BTC Trade Execution:**

After signals, if `edgeResult.tradeable` **and** no suppression flags (warmup, trend, distance, etc.), it runs `KellySizer`, and if `stake>0`, does `insertPaperTrade.run(...)` and sends a Telegram alert. Otherwise, it logs “skipping trade” details but does not trade.

- **ETH Processing:**

After BTC, the code does *ETH market processing* (lines ~315 onward). Key branches:

- It checks `if (ethData && ethData.priceUp && ethData.priceDown)`. If PolymarketEth data is incomplete, it would skip ETH.
- It computes `const ethBayes = this.ethBayesian.estimate(...); if (!ethBayes) return;` (**IMPORTANT**). This `return` is inside `_onPriceUpdate`, so it aborts the function entirely. **Thus, if ETH K is stale, the BTC post-processing (which is already done) is not affected, but any code after this ETH block in the same tick is skipped.**
- If `ethBayes` exists, it mirrors BTC: `const ethEdge = this.edge.evaluate(...)`, logs an ETH signal via `insertSignal`, and conditionally executes an ETH trade with `KellySizer`.

Because ETH processing is at the end of the function, that `return` on stale ETH only prevents any code after it (e.g. final statistics or Ethereum-specific trade logic), but since BTC is already done, it mostly just skips ETH trade that tick. This asymmetry should be noted (see fixes below).

BTC vs ETH Code Path Differences

Aspect	BTC Path	ETH Path
Initial Data	Uses <code>rtdsState.chainlinkPrice</code> , <code>rtdsState.chainlinkAge</code> , <code>rtdsState.binanceVelocity</code> , and <code>polyData</code> from <code>this.lastPolyData</code> .	Uses <code>rtdsState.ethChainlinkPrice</code> , <code>rtdsState.ethChainlinkAge</code> , <code>rtdsState.ethBinanceVelocity</code> , and <code>ethData</code> from <code>this.lastEthData</code> .
Bayesian Estimator	Single <code>this.bayesian</code> instance. Sets <code>openingPrice</code> on BTC windows.	Separate <code>this.ethBayesian</code> . Sets its own <code>openingPrice</code> on ETH windows.
Window Ts	Uses <code>polyData.windowTs</code> (BTC market slug).	Uses <code>ethData.windowTs</code> (ETH market slug).
Suppression Flags	Builds <code>suppressionFlags</code> array of 5 conditions (warmup, win_lock, trend, gap, dist).	Builds <code>ethSuppressionFlags</code> array of 4 conditions (warmup, win_lock, dist, trend). But code fails to use it in DB insert (bug).
Signal Insert Columns	Inserts <i>all</i> columns (timestamp, slug, seconds_left, prices, K info, edge, flags, etc.) – see extract below.	(Bug) The <code>insertSignal</code> call ends prematurely (at line 420) missing many columns (post_vacuum, vacuum_age, suppression, imbalance, etc.).

Aspect	BTC Path	ETH Path
Trade Execution	If <code>edgeResult.tradeable && ! suppressed</code> , does Kelly sizing and <code>insertPaperTrade</code> .	If <code>ethEdge.tradeable && ! ethSuppressed</code> , does Kelly sizing and <code>insertPaperTrade</code> . Uses <code>ethSlug</code> and <code>ethData.priceUp/down</code> .
Up-Diagnostics (UP-only)	Logs pure-up arbitrages to <code>up_diagnostics</code> when <code>edge.direction==='UP' && secsLeft>=15</code> . Adds rows with <code>suppression_reason</code> .	No equivalent ETH up-diagnostics (or it was omitted).

Signal Insert (BTC) Example:

```
INSERT INTO signals (
  timestamp, window_slug, seconds_left,
  binance_price, chainlink_price, binance_gap_pct, lag_detected, lag_direction,
  btc_velocity, chainlink_age_ms,
  poly_price_up, poly_price_down, spread, bid_depth, ask_depth, liquidity,
  opening_price, fair_prob_up, distance_strike, volatility, drift, observations,
  raw_edge, net_edge, total_cost, direction, tradeable,
  filter_failures, alert_sent, post_vacuum, vacuum_age_s, suppression_flags,
  imbalance_ratio, imbalance_delta, imbalance_bucket,
  would_trade_down_imbalance, would_trade_up_imbalance, near_threshold,
  raw_edge_pct, final_edge_pct, trend_penalty, trend_would_block
) VALUES ( ... );
```

All 40+ fields are populated for BTC. **ETH insertion in code only passes the first ~25 fields (up to `lag_direction` etc.) and then abruptly ends.** This mismatch must be corrected (see Fix #1).

Suppression Flags vs Execution

- In the BTC path, the code computes a `suppressionFlags` string (e.g. `'trend,dist_from_k'`) and passes it into the `INSERT signals` for `suppression_flags`. The trade is executed only if none of these are true.
- In the ETH path, a similar array `ethSuppressionFlags` is built, but **never used** in the insert (as noted above). Also, ETH's Tier determination and suppression logic uses a variable `ethSuppressed` separately. This inconsistency means ETH trades may execute differently than BTC.

Recommendation: Use the same pattern for ETH – i.e., compute `ethSuppressionFlags` and include it in the DB insert. This will let post-mortem analysis treat BTC/ETH similarly.

Windowing Logic

- A new 5-minute window is detected when `polyData.windowTs` changes. On rollover, the engine resolves the previous window (if possible) and resets Bayesian/spread models.
- **Rollover Race:** The code sets `this.lastWindowTs = windowTs` *after* resolving the old window. This prevents double-resolving on multiple ticks within the same new window. This logic appears correct.
- **Potential desync:** If PolymarketFetcher's `windowTs` lags RTDS updates, the “first” tick might belong to the old window or vice versa. The code does *not* validate that `chainlinkPrice` is exactly at window boundary. As a precaution, it logs a warning and defers resolution if Chainlink is stale (lines 104–112). This was the intended “fix” for late Chainlink, but our review suggests it may not have been triggered in reality (see STALE-K note).

In short, window boundaries depend entirely on Polymarket data (`windowTs`), and the code trusts it. We suggest logging any discrepancy between `secsLeft` and change of window, but no immediate fix needed.

Logging Coverage

- The code logs most key events:
- **Window roll:** `"[Engine] □ Window rolled: ..."`
- **Deferred resolution:** `"[Engine] ⚠ Resolution deferred..."`
- **Balance updates.**
- **Vacuum start/end.**
- **Each executed trade** (via `[Paper] Trade logged`) and Telegram alerts.
- **Missing logs:**
- **Tick skips:** No log when a tick is skipped due to `!polyData`, `!chainlinkPrice`, or stale-K (besides the K-Guard log in Bayesian). We recommend adding a console log for each such skip.
- **Errors:** Only a generic `console.error` on catch. Consider adding context (which tick or values).

Adding a few targeted log lines (e.g. on each `return` path) will greatly improve observability.

Issues & Recommendations (with Code Snippets)

Below we list each notable issue found, its impact, and a suggested code fix.

1. ETH `signals` Insert Missing Fields (Critical)

Issue: The `insertSignal.run(...)` call for ETH (around lines 381–420 in `arb_engine.js`) **only provides a handful of columns** and then terminates. In particular, it does not supply `suppression_flags`, imbalance fields, or the trailing edge percentages. This will cause the `signals` table to have misaligned columns for ETH rows.

Severity: Critical (database misalignment & missing diagnostics).

Location: `arb_engine.js` in the ETH block
(`if (ethEdge.tradeable) { ... const ethSignalRow = insertSignal.run( ... ); }`).

Reproduction: Inspect the ETH `signals` rows in the DB after a run; many columns will be null or misordered. A quick check:

```
SELECT * FROM signals WHERE window_slug LIKE 'eth-updown-5m-%' LIMIT 1;
```

Compare with the inserted columns for BTC.

Recommended Fix: Supply the remaining values just as done for BTC. For example (showing the appended arguments in `insertSignal.run`):

```
--- a/arb_engine.js
+++ b/arb_engine.js
@@ (ETH insertSignal.run parameters)
     ethEdge.tradeable ? 1 : 0,
     ethEdge.netEdge,
     0 // no ETH lag detection
-   /* missing fields here */
+   , 0 // alert_sent
+   , ethEdge.postVacuum ? 1 : 0
+   , ethEdge.postVacuum ? ((Date.now() - this.lastVacuumTime)/1000) : null
+   , ethSuppressionFlags
+   , ethImbalanceRatio, ethImbalanceDelta, ethImbalanceBucket
+   , ethWouldTradeDown, ethWouldTradeUp, ethNearThreshold
+   , ethEdge.netEdge * 100 // raw_edge_pct (before trend)
+   , (ethEdge.netEdge - (ethTrendPenalty||0)) * 100 // final_edge_pct
+   , ethTrendPenalty, ethTrendBlocked ? 1 : 0
  );
```

Adjust exactly to match the `INSERT signals` schema. (Ensure commas and parentheses align.)

2. Chainlink Age Default and Guard (High)

Issue: In `bayesian.js`, the code calls `recordOpeningPrice(currentPrice, windowTs, chainlinkAgeMs ?? 0)`. If `chainlinkAgeMs` is `undefined` (e.g. not passed), it becomes `0` by default, bypassing the stale-K guard. This hides missing data.

Severity: High (could silently accept stale or missing data as fresh).

Location: `bayesian.js`, in `estimate()` (lines 170–180 of that file).

Reproduction: Instrument RTDS to drop the `chainlinkAge` field. Without fix, the guard never triggers. Alternatively, scan logs to see if any “[K-Guard]” messages ever appear – they won’t if age was undefined.

Recommended Fix: Require an explicit age. For example, modify the `estimate` call in `arb_engine.js` to always pass `rtdsState.chainlinkAge ?? null`, and in `bayesian.js`:

```
--- a/bayesian.js
+++ b/bayesian.js
  estimate(currentPrice, secondsRemaining, velocityPerSec, windowTs,
  chainlinkAgeMs) {
-    if (this.openingWindowTs !== windowTs || this.openingPrice === null) {
-      const ok = this.recordOpeningPrice(currentPrice, windowTs,
  chainlinkAgeMs ?? 0);
+    if (this.openingWindowTs !== windowTs || this.openingPrice === null) {
+      if (chainlinkAgeMs === undefined) {
+        console.warn(`[K-Guard] Missing chainlinkAgeMs for window $
  {windowTs}`);
+      }
+      const ok = this.recordOpeningPrice(currentPrice, windowTs,
  chainlinkAgeMs);
      if (!ok || this.openingPrice == null) {
        return null;
      }
    }
  }
```

And in `recordOpeningPrice`, no change (it already rejects >5min). This ensures if age is truly absent, we log it.

3. Return Behavior on ETH Stale K (High)

Issue: The line `if (!ethBayes) { return; }` (around line 316 of `arb_engine.js`) sits inside the main async function. A `return` here exits `_onPriceUpdate` entirely, preventing any code after (like final logs or any other processing) from running. It also means that any catching outside might not see this as an error.

Severity: High (unexpected control flow; could skip important logic).

Location: `arb_engine.js`, ETH section after `const ethBayes = this.ethBayesian.estimate(...)`.

Reproduction: Simulate an ETH stale-K (e.g. by forcing a stale-chainlink update). Observe that no Ethereum trade occurs but also that any code after this point is skipped.

Recommended Fix: Limit the skip to ETH only. Instead of `return`, simply avoid the rest of ETH logic. For example:

```

- if (!ethBayes) {
-   return; // stale K – skip this ETH tick
- }
+ if (!ethBayes) {
+   console.log(`[Engine] ETH tick skipped: stale K (age $
{rtdsState.ethChainlinkAge}ms)`);
+   // Do not return the whole function; skip only ETH logic
+ } else {
+   // (Wrap the rest of ETH processing code in here)
+   const ethEdge = this.edge.evaluate(ethBayes, ethData);
+   ...
+ }

```

Or restructure `async _onPriceUpdate` so BTC and ETH blocks are separate try/catch scopes. The key is to avoid aborting all processing due to ETH.

4. No-Price Early Return (Medium)

Issue: The code does `if (!chainlinkPrice) return;` without logging. This silently skips ticks if the Chainlink feed is momentarily empty.

Severity: Medium.

Location: `arb_engine.js`, around line 515.

Reproduction: Temporarily disable Chainlink updates. The engine will quietly do nothing each tick (no log).

Recommended Fix: Add a log or warning for visibility:

```

- if (!chainlinkPrice) return;
+ if (!chainlinkPrice) {
+   console.log(`[Engine] Skipping tick ${this.tickCount}: BTC Chainlink price
missing`);
+   return;
+ }

```

This will help diagnose feed interruptions.

5. ETH Suppression Flags Not Used (Medium)

Issue: The ETH code builds `ethSuppressionFlags` but never passes it to `insertSignal`. BTC uses `suppressionFlags`.

Severity: Medium.

Location: `arb_engine.js`, lines 374–379 (flag definition) and signal insert.

Reproduction: ETH signal rows in DB will have `suppression_flags='none'` even if a suppression applied.

Recommended Fix: In the ETH `insertSignal.run` parameters, use `ethSuppressionFlags` in the `suppression_flags` column. Example (as in fix #1):

```
@@ ... , ethEdge.postVacuum ? ((Date.now()-this.lastVacuumTime)/1000) : null,  
-   ethImbalanceRatio, ethImbalanceDelta, ethImbalanceBucket,  
+   ethImbalanceRatio, ethImbalanceDelta, ethImbalanceBucket,  
+   ethSuppressionFlags,  
    ethWouldTradeDown, ethWouldTradeUp, ethNearThreshold,  
    ethEdge.netEdge * 100, (ethEdge.netEdge - ethTrendPenalty)*100,  
    ethTrendPenalty, ethTrendBlocked ? 1 : 0  
);
```

(Place it right before or after imbalance fields to match the schema.)

6. Vacuum Flags for ETH (Medium)

Issue: The BTC insert uses `isPostVacuum` and vacuum age; ETH insert currently passes `0` for lag detection but not for vacuum.

Severity: Medium.

Location: `arb_engine.js`, ETH signal insert (lines ~381–420).

Reproduction: The `post_vacuum` and `vacuum_age_s` columns in ETH signals remain default/null, losing information on whether the trade occurred right after a liquidity vacuum.

Recommended Fix: Mirror BTC's logic. Define `const ethPostVacuum = ...` and include it. For example:

```
... , 0 // no ETH lag detection  
+   , ethEdge.postVacuum ? 1 : 0  
+   , ethEdge.postVacuum ? ((Date.now() - this.lastVacuumTime)/1000) : null  
+   , ethSuppressionFlags, ...
```

Make sure column order matches the schema.

7. WindowTs Validation (Medium)

Issue: No check ensures the tick actually belongs to the announced window. E.g., an RTDS update could arrive just before/after a window boundary. The code uses `windowTs` directly for K and rolling.

Severity: Medium.

Location: `arb_engine.js`, rollover block (lines 100–150).

Reproduction: Cross-check `rtDsState.chainlinkPrice` timestamp vs. `polyData.windowTs` change. If inconsistent, alerts should trigger.

Recommended Fix (diagnostic): After rollover, compare `secondsRemaining` or actual time. Example log:

```
if (this.lastWindowTs && this.lastWindowTs !== windowTs) {
  if (secsLeft > 290) {
    console.warn(`[Engine] Window rolled late: previous window $
    {this.lastWindowTs} had ${secsLeft}s left`);
  }
  // existing rollover logic...
}
```

This does not fix logic but would highlight any misalignment. If misalign is found, one might adjust `bayesian` to hold off setting K until `secsLeft` crosses the boundary, but that's complex.

8. Inconsistent Time Units (Low)

Issue: `windowTs` (from Polymarket) and the code's `Date.now()` must both be in milliseconds epoch. It is likely, but not enforced.

Severity: Low.

Location: All time usage.

Recommended Action: Verify PolymarketFetcher's `windowTs` unit. If in seconds, convert to ms. For safety, add a comment or convert:

```
const windowTs = polyData.windowTs * 1000; // if provided in seconds
```

Also, clarify in logs (e.g. `new Date(windowTs).toISOString()`). This is a documentation fix more than code.

9. Add Logs on Skips (Low)

Issue: Early `return` statements (pause, no data, stale K) have no corresponding log.

Severity: Low.

Location: Various lines with `return`.

Recommended Fix: Insert simple logs. For example:

```
// After if (this.paused)
+ console.log(`[Engine] Skipped tick ${this.tickCount}: engine paused`);
```

And similarly for missing `polyData` or `bayesResult`. Ensure logs do not flood – you may want to filter in practice.

10. Document Pre/Post-Penalty in DB (Low)

Issue: The meaning of stored edge values should be clear. Currently, `net_edge_pct` in `paper_trades` is saved before trend penalty, but this is not obvious.

Severity: Low.

Location: `arb_engine.js`, `insertPaperTrade` call.

Recommended Fix: Add an inline comment. For example:

```
insertPaperTrade.run(
  timestamp, slug, direction, entryPrice, stake, shares,
  kellyFraction, edgeResult.netEdge * 100, // net_edge_pct (before trend
  penalty)
  ...
);
```

Or store both values if desired. This is mostly documentation.

<div style="page-break-after: always;"></div>

Comparison: BTC vs ETH Paths

Component	BTC Version	ETH Version
Price Data	<code>rtdsState.chainlinkPrice,</code> <code>rtdsState.chainlinkAge,</code> <code>rtdsState.binancePrice,</code> <code>rtdsState.binanceVelocity.</code>	<code>rtdsState.ethChainlinkPrice,</code> <code>rtdsState.ethChainlinkAge,</code> <code>rtdsState.ethBinancePrice,</code> <code>rtdsState.ethBinanceVelocity.</code>
Polymarket Fetch	<code>this.polymarket.fetchMarketPrice()</code> , stored in <code>this.lastPolyData</code> . Window slug for BTC Up/Down.	<code>this.polymarketEth.fetchMarketPrice()</code> , stored in <code>this.lastEthData</code> . Window slug for ETH Up/Down.
Bayesian Instance	<code>this.bayesian; fed (chainlinkPrice, secsLeft, binanceVelocity, windowTs, chainlinkAge).</code>	<code>this.ethBayesian; fed (ethCL, ethData.secondsRemaining, ethBinanceVelocity, ethData.windowTs, ethChainlinkAge).</code>
Edge Filtering	<code>const edgeResult = this.edge.evaluate(bayesResult, polyData).</code>	<code>const ethEdge = this.edge.evaluate(ethBayes, ethData).</code> Same edge model used.
Suppression Conditions	Conditions: not warmed up, window lock, trend, gap, distance from K (<code>suppressionFlags</code>). Trade only if all false.	Conditions: not warmed up, window lock, trend, distance from K (<code>ethSuppressionFlags</code>). No ETH "gap" flag used (lag unknown).
Signal Insert	All fields populated, including <code>suppressionFlags</code> and imbalance metrics. Triggers update of <code>signals</code> table.	Bug: Only initial 25 fields are provided. Missing vacuum, suppression, imbalance, etc.
Trade Sizing	KellySizer with tier caps, uses <code>edgeResult.direction ? priceUp/priceDown</code> . Trades USD.	Similarly uses <code>ethEdge.direction ? ethData.priceUp/priceDown</code> . Trades USD.
Paper Trade Insert	Inserts: timestamp, slug, direction, entry, stake, shares, kelly%, net_edge%, balance, tier, up-signal context.	Identical pattern, using <code>ethSlug</code> , <code>ethData.priceUp/down</code> . (Seen in lines 447-459.)

Logging Gaps and Observability Suggestions

- **Tick Skips:** Add logs for `return` paths (pause, no data, stale K).
- **K-Guard Events:** We already log `[K-Guard] Skipped stale first tick...`. Also log a message whenever `bayesian.estimate()` returns null.
- **Resolution Events:** The rollover resolution logs the trend (good), but perhaps log the pending resolution in more detail.
- **Alert Logging:** After sending Telegram, the code updates `alert_sent`. Could log if `sendTelegram` fails.

Action Items (Prioritized)

1. **Fix ETH signal insertion (Critical):** Complete the `INSERT signals` for ETH with all columns (as above). (2h)
2. **Fix chainlinkAge handling (High):** Ensure age is correctly passed and logged. (1h)
3. **Change ETH `return` to skip-only (High):** Prevent aborting entire tick on ETH stale-K. (2h)
4. **Add missing logs (Medium):** Log all skip paths and important checks. (1h)
5. **Use ETH suppressionFlags (Medium):** Pass `ethSuppressionFlags` into ETH signals insert. (30m)
6. **Add ETH vacuum flags (Medium):** Populate `post_vacuum` and `vacuum_age_s` for ETH signals. (30m)
7. **Verify time units (Low):** Confirm `windowTs` units; convert if needed, document. (30m)
8. **Document signal/trade captures (Low):** Add comments clarifying pre/post-penalty columns. (30m)
9. **Review spread_model usage:** Not shown here, but ensure it's not gating trades incorrectly (diagnostic). (30m)
10. **Write DB-check queries (Low):** For QA, use SQL to verify every column is populated for both BTC and ETH. (Not code, but a recommended check.)

The **major corrections** (items 1–3) should be done first. They directly fix data corruption and control flow bugs. Lesser tasks (4–10) improve robustness and debuggability.

Overall, once these fixes are applied, the engine's data pipeline will be **much more reliable and transparent**. All primary outputs (`signals` and `paper_trades`) will reflect the true intended logic, and future anomalies will be easier to trace.
